UNIT -II

REVIEW OF CFG AMBIGUITY OF GRAMMARS

LECTURE NO: 1

Derivation

A **derivation** is a sequence of replacements of structure names by choices on the right-hand sides of grammar rules.

A **derivation** begins with a single structure name and ends with a string of token symbols.

At each step in a derivation, a single replacement is made using one choice from a grammar rule.

exp → *exp op exp* | *(exp)* | ***number***
op → + | – | *

Example 1 : a derivation

(1) *exp* => *exp op exp* [*exp* → *exp op exp*]
(2) => *exp op* ***number*** [*exp* → number]
(3) => *exp* * ***number*** [*op*→ *]
(4) => (*exp*) * ***number*** [*exp*→ (*exp*)]
(5) =>{ *exp op exp*) * ***number*** [*exp* → *exp op exp}*
(6) => *(exp op* number) * ***number*** [*exp*→ number]
(7) => *(exp* - number) * ***number*** [*op* → -]
(8) => (number - number) * ***number*** [*exp* → number]

derivation steps use a different arrow from the arrow meta-symbol in the grammar rules. Because grammar rules define and derivation steps construct by replacement.

L(G) = { *s* | *exp* =>* *s* }

The set of all strings of token symbols obtained by derivations from the *exp* symbol is the **language defined by the grammar** of expressions.

(1) *G* represents the expression grammar
(2) *s* represents an arbitrary string of token symbols (sometimes called a sentence)
(3) The symbols =>* stand for a derivation consisting of a sequence of replacements as described earlier. (The asterisk is used to indicate a sequence of steps, much as it indicates repetition in regular expressions.)
(4) Grammar rules are sometimes called productions because they "produce" the strings in *L(G)* via derivations.

Example 2: A CFG for **ab* = { a, ab, abb, abbb, abbbb, ... }**

1. Terminals: ∑ = {a, b},
2. Nonterminal: N = {S, B}
3. Productions: P = {

S -> aB
B -> bB | ^
}

S => aB => abB => abbB => abbbB => abbb

Most of the time the set of productions are explicitly given for a CFG and the terminals and the non-terminals are understood from the context as shown in examples 4-6 :

Example 3: A CFG for **aba* = { ab, aba, abaa, abaaa, ... }**

S -> abA
A -> aA | ^

S => abA => abaA => abaaA => abaaaA => abaaa

Example 4: A CFG for { $\mathbf{a^n cb^n : n > 0}$ } **= { acb, aacbb, aaacbbb, ... }**

S -> aSb | acb

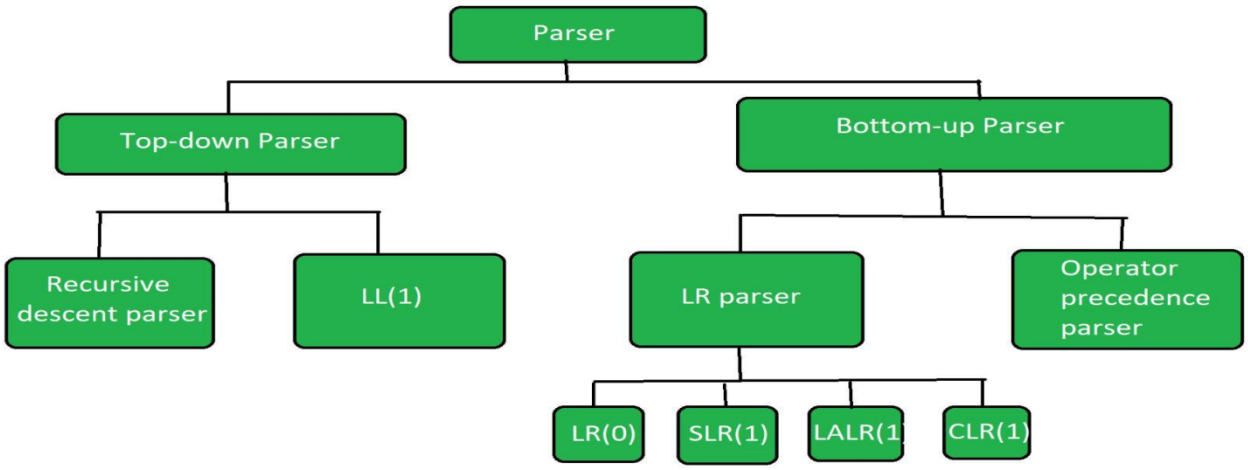
DERIVATION of **aaacbbb** is,

S => aSb => aaSbb => aaacbbb

LECTURE 2

INTRODUCTION TO PARSING

Parser is that phase of compiler which takes token string as input and with the help of existing grammar, converts it into the corresponding parse tree. Parser is also known as Syntax Analyzer.



Types of Parsers

Parser is mainly classified into 2 categories: Top-down Parser, and Bottom-up Parser. These are explained as following below.

TOP-DOWN PARSER

Top-down parser is the parser which **generates parse for the given input string** with the help of grammar productions by expanding the non-terminals i.e., it starts from the start symbol and ends on the terminals. it uses left derivation.

Further Top-down parser is classified into 2 types: Recursive descent parser, and Non-recursive descent parser.

(i) Recursive descent parser

It is also known as Brute force parser or the with backtracking parser. It basically generates the parse tree by using brute force and backtracking.

(ii) non recursive descent parser

It is also known as LL(1) parser or predictive parser or without backtracking parser or dynamic parser. It uses parsing table to generate the parse tree instead of backtracking.

2. Bottom-up parser

Bottom-up Parser is the parser which generates the parse tree for the given input string with the help of grammar productions by compressing the non-terminals i.e. it starts from non-terminals and ends on the start symbol.It uses reverse of the right most derivation. Further Bottom-up parser is classified into 2 types: LR parser, and Operator precedence parser.

LR PARSER

LR parser is the bottom-up parser which generates the parse tree for the given string by using unambiguous grammar It follow reverse of right most derivation. LR parser is of 4 types:

(a). LR (0)

(b). SLR (1)

(c). LALR (1)

(d). CLR (1)

There are 2 types of Parsing Technique present in parsing, first one is Top-down parsing and second one is Bottom-up parsing.

Top-down Parsing is a parsing technique that first looks at the highest level of the parse tree and works down the parse tree by using the rules of grammar while **Bottom-up Parsing** is a parsing technique that first looks at the lowest level of the parse tree and works up the parse tree by using the rules of grammar.

S.No	TOP-DOWN PARSING	BOTTOM-UP PARSING
1.	It is a parsing strategy that first looks at the highest level of the parse tree and works down the parse tree by using the rules of grammar.	It is a parsing strategy that first looks at the lowest level of the parse tree and works up the parse tree by using the rules of grammar.

2.	Top-down parsing attempts to find the left most derivations for an input string.	Bottom-up parsing can be defined as an attempt to reduce the input string to start symbol of a grammar.
3.	In this parsing technique we start parsing from top (start symbol of parse tree) to down (the leaf node of parse tree) in top-down manner.	In this parsing technique we start parsing from bottom (leaf node of parse tree) to up (the start symbol of parse tree) in bottom-up manner.
4.	This parsing technique uses Left Most Derivation.	This parsing technique uses Right Most Derivation.
5.	It's main decision is to select what production rule to use in order to construct the string.	It's main decision is to select when to use a production rule to reduce the string to get the starting symbol.

LECTURE 3

SHIFT REDUCE PARSING AND OPERATOR PRECEDANCE PARSING

SHIFT REDUCE PARSING

Shift Reduce parser attempts for the construction of parse in a similar manner as done in bottom up parsing i.e. the parse tree is constructed from leaves(bottom) to the root(up). A more general form of shift reduce parser is LR parser.
This parser requires some data structures i.e.

- A input buffer for storing the input string.
- A stack for storing and accessing the production rules.

Basic Operations –

- **Shift:** This involves moving of symbols from input buffer onto the stack.
- **Reduce:** If the handle appears on top of the stack then, its reduction by using appropriate production rule is done i.e. RHS of production rule is popped out of stack and LHS of production rule is pushed onto the stack.
- **Accept:** If only start symbol is present in the stack and the input buffer is empty then, the parsing action is called accept. When accept action is obtained, it is means successful parsing is done.
- **Error:** This is the situation in which the parser can neither perform shift action nor reduce action and not even accept action.

EXAMPLE:1

S–>S+S
S–>S*S
S –> id

Perform Shift Reduce parsing for input string “id + id + id”.

EXAMPLE: 2

E–>2E2
E–>3E3
E–>4

Perform Shift Reduce parsing for input string “32423”.

Stack	Input Buffer	Parsing Action
$	32423$	Shift
$3	2423$	Shift
$32	423$	Shift
$324	23$	Reduce by E --> 4
$32E	23$	Shift
$32E2	3$	Reduce by E --> 2E2
$3E	3$	Shift
$3E3	$	Reduce by E --> 3E3
$E	$	Accept

EXAMPLE: 3

S–> (L)|a
L–> L,S|S

Perform Shift Reduce parsing for input string "(a , (a , a)) ".

Stack	Input Buffer	Parsing Action
$	(a , (a , a)) $	Shift
$ (	a , (a , a)) $	Shift
$ (a	, (a , a)) $	Reduce S → a
$ (S	, (a , a)) $	Reduce L → S
$ (L	, (a , a)) $	Shift
$ (L ,	(a , a)) $	Shift
$ (L , (	a , a)) $	Shift
$ (L , (a	, a)) $	Reduce S → a
$ (L , (S	, a)) $	Reduce L → S

$ (L , (L	, a)) $	Shift
$ (L , (L ,	a)) $	Shift
$ (L , (L , a	)) $	Reduce S → a
$ (L , (L , S)	)) $	Reduce L →L , S
$ (L , (L	)) $	Shift
$(L,(L)	)$	Reduce S->(L)
$(L,S)	$	ReduceL->L,S
$(L)	$	Reduce S->(L)
$ S	$	Accept

Operator precedence parsing

Operator precedence grammar is kinds of shift reduce parsing method. It is applied to a small class of operator grammars.

A grammar is said to be operator precedence grammar if it has two properties:

- No two non-terminals are adjacent
- No R.H.S. of any production has a∈.

Operator precedence can only establish between the terminals of the grammar. It ignores the non-terminal.
There are the three operator precedence relations:

a ⋗ b means that terminal "a" has the higher precedence than terminal "b".

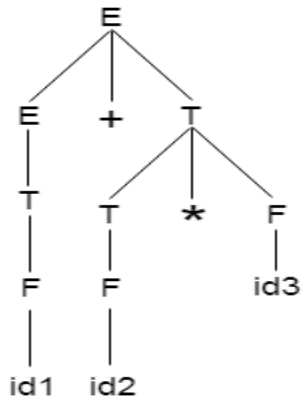
Grammar:

1. E → E+T/T
2. T → T*F/F
3. F → id

Given string:

1. w = id + id * id

Let us consider a parse tree for it as follows:


E
E
+
T
T
T
*
F
F
F
id3
id1
id2

LECTURE NO:4

RECURSIVE DESCENT PARSING

Predictive Parser - Recursive Descent Parser (RDP)

A parser that uses a set of recursive procedures to recognize its input with no backtracking is called a recursive descent parser For implementing a recursive descent parser for a grammar.

- The grammar must not be left recursive
- The grammar must be left factored that means it should not have common prefixes for alternates
- We need a language that has recursion facility

Left factoring Problems:

Eliminate Left recursion and then left factor the following grammar

rexpr → reexpr + rterm | rterm
rterm → rterm rfactor | rfactor
rfactor → rfactor* | rprimary
rprimary → a | b

Design:

- It can be written on the same lines as that of brute force approach that is one procedure for one non terminal
- The procedures need not return anything because they need not try another alternate as in brute force
- Only on failure we call some error routine to take care of the error

On can make use of different type of transition diagrams for designing a recursive descent parser as used in case of scanners.

The differences are

- There is one transition diagram for one non terminal
- The labels for edges may be terminals or non-terminals
- Label is a terminal which means we must take that transition if the next input symbol is terminal

After elimination of left recursion and left factoring one can construct the transition diagram for the grammar as, For each non-terminal A, Create an initial and final states

- For each production $A \rightarrow X_1, X_2,....X_n$, create a path from initial to the final state with edges labeled $X_1, X_2, ... X_n$

Advantage:

- Easy to hand construct a recursive descent parser from an appropriate grammar

Limitations

- Recursive descent parser encodes state information in its run-time stack, or call stack, and this may not be particularly efficient
- Uses recursive procedure calls to implement a stack abstraction
- It is very difficult to automated.

LECTURE NO-5

PREDICTIVE PARSER AND LL GRAMMAR

Predictive Parser in Compiler Design

In this, we will cover the overview of Predictive Parser and mainly focus on the role of Predictive Parser. And will also cover the algorithm for the implementation of the Predictive parser algorithm and finally will discuss an example by implementing the algorithm for precedence parsing. Let's discuss it one by one.

Predictive Parser:

A predictive parser is a recursive descent parser with no backtracking or backup. It is a top-down parser that does not require backtracking. At each step, the choice of the rule to be expanded is made upon the next terminal symbol.

Consider
A -> A1 | A2 | ... | An

If the non-terminal is to be further expanded to 'A', the rule is selected based on the current input symbol 'a' only.

Predictive Parser Algorithm:

1. Make a transition diagram (DFA/NFA) for every rule of grammar.
2. Optimize the DFA by reducing the number of states, yielding the final transition diagram.
3. Simulate the string on the transition diagram to parse a string.
4. If the transition diagram reaches an accept state after the input is consumed, it is parsed.

Consider the following grammar –

E->E+T|T

T->T*F|F

F->(E)|id

After removing left recursion, left factoring

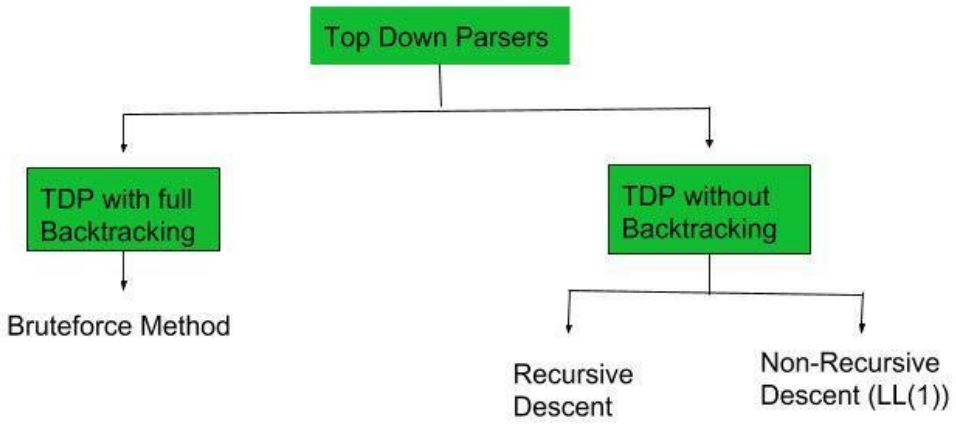
E->TT'
T'->+TT'|ε
T->FT"
T"->*FT"|ε
F->(E)|id

Construction of LL(1) Parsing Table

A top-down parser builds the parse tree from the top down, starting with the start non-terminal. There are two types of Top-Down Parsers:

1. Top-Down Parser with Backtracking
2. Top-Down Parsers without Backtracking

Top-Down Parsers without backtracking can further be divided into two parts:



In this article, we are going to discuss Non-Recursive Descent which is also known as LL(1) Parser.

LL(1) Parsing:

Here the 1st **L** represents that the scanning of the Input will be done from Left to Right manner and the second **L** shows that in this parsing technique we are going to use Left most Derivation Tree. And finally, the **1** represents the number of look-ahead, which means how many symbols are you going to see when you want to make a decision.

Algorithm to construct LL(1) Parsing Table:

Step 1: First check for left recursion in the grammar, if there is left recursion in the grammar remove that and go to step 2.

Step 2: Calculate First() and Follow() for all non-terminals.
1. First (): If there is a variable, and from that variable, if we try to drive all the strings then the beginning Terminal Symbol is called the First.
2. Follow(): What is the Terminal Symbol which follows a variable in the process of derivation.

Step 3: For each production A –> α. (A tends to alpha)
1. Find First(α) and for each terminal in First(α), make entry A –> α in the table.
2. If First(α) contains ε (epsilon) as terminal than, find the Follow(A) and for each terminal in Follow(A), make entry A –> α in the table.

3. If the First(α) contains ε and Follow(A) contains $ as terminal, then make entry A –> α in the table for the $. To construct the parsing table, we have two functions:

In the table, rows will contain the Non-Terminals and the column will contain the Terminal Symbols. All the **Null Productions** of the Grammars will go under the Follow elements and the remaining productions will lie under the elements of the First set.
Now, let's understand with an example.

Example-1:
Consider the Grammar:
E --> TE'

E' --> +TE' | ε

T --> FT'

T' --> *FT' | ε

F --> id | (E)

*ε denotes epsilon

Find their First and Follow sets:

	First	Follow
E –> TE'	{ id, (}	{ $,) }
E' –> +TE'/ε	{ +, ε }	{ $,) }
T –> FT'	{ id, (}	{ +, $,) }
T' –> *FT'/ε	{ *, ε }	{ +, $,) }
F –> id/(E)	{ id, (}	{ *, +, $,) }

Now, the LL(1) Parsing Table is:

	Id	+	*	(	)	$
E	E –> TE'			E –> TE'		
E'		E' –> +TE'			E' –> ε	E' –> ε
T	T –> FT'			T –> FT'		
T'		T' –> ε	T' –> *FT'		T' –> ε	T' –> ε
F	F –> id			F –> (E)		

As you can see that all the null productions are put under the Follow set of that symbol and all the remaining productions are lie under the First of that symbol.

Note: Every grammar is not feasible for LL(1) Parsing table. It may be possible that one cell may contain more than one production.

Let's see with an example.

Example-2:
Consider the Grammar

S --> A | a

A --> a

Find their First and Follow sets:

	First	Follow
S –> A/a	{ a }	{ $ }
A –>a	{ a }	{ $ }

Parsing Table:

	A	$
S	S –> A, S –> a	
A	A –> a	

Here, we can see that there are two productions into the same cell. Hence, this grammar is not feasible for LL(1) Parser.

LECTURE NO-6
ERROR HANDLING OF LR PARSER

The tasks of the **Error Handling** process are to detect each error, report it to the user, and then make some recovery strategy and implement them to handle the error. During this whole process processing time of the program should not be slow.
Functions of Error Handler:

- Error Detection
- Error Report
- Error Recovery

Error handler=Error Detection Error Report Error Recovery.

An **Error** is the blank entries in the symbol table

Errors in the program should be detected and reported by the parser. Whenever an error occurs, the parser can handle it and continue to parse the rest of the input. Although the parser is mostly responsible for checking for errors, errors may occur at various stages of the compilation process. So, there are many types of errors and some of these are:

Types or **Sources of Error** – There are three types of error: logic, run-time and compile-time error:

1. **Logic errors** occur when programs operate incorrectly but do not terminate abnormally (or crash). Unexpected or undesired outputs or other behaviour may result from a logic error, even if it is not immediately recognized as such.

2. A **run-time error** is an error that takes place during the execution of a program and usually happens because of adverse system parameters or invalid input data. The lack of sufficient memory to run an application or a memory conflict with another program and logical error is an example of this. Logic errors occur when executed code does not produce the expected result. Logic errors are best handled by meticulous program debugging.

3. **Compile-time errors** rise at compile-time, before the execution of the program. Syntax error or missing file reference that prevents the program from successfully compiling is an example of this.

IV) Error Production



- A method of predict common errors that might be encountered.
- Augmenting the grammar for the language at hand, with productions as : **A-> Error.**
- Such a parser will detect expected errors when an error production is used.
- **Ex:- Automatic Error recovery in YACC**

  Use error production with semantic actions

  A : Error ε {semantic action to recover error}.

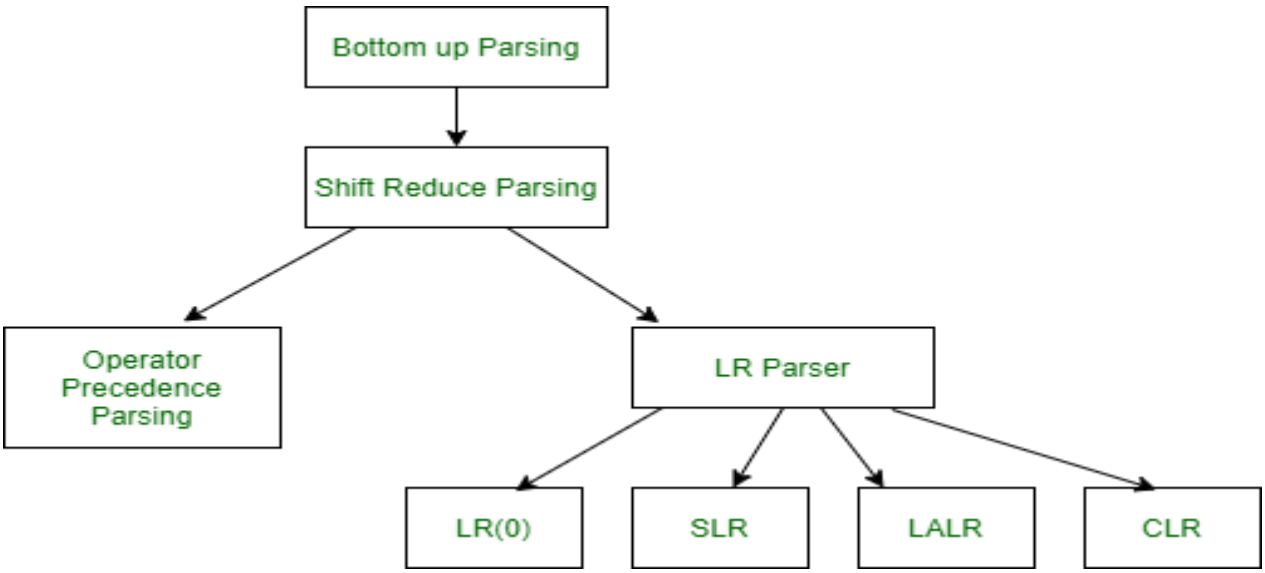
- **Advantage: Error diagnostics is very fast.**

LECTURE NO-7

LR PARSER AND CONSTRUCTION OF SLR

LR parser is a bottom-up parser for context-free grammar that is very generally used by computer programming language compiler and other associated tools.

LR parser reads their input from left to right and produces a right-most derivation. It is called a Bottom-up parser because it attempts to reduce the top-level grammar productions by building up from the leaves. LR parsers are the most powerful parser of all deterministic parsers in practice.



Description of LR PARSER

The term parser LR(k) parser, here the L refers to the left-to-right scanning, R refers to the rightmost derivation in reverse and k refers to the number of unconsumed "look ahead" input symbols that are used in making parser decisions. Typically, k is 1 and is often omitted.

A context-free grammar is called LR (k) if the LR (k) parser exists for it. This first reduces the sequence of tokens to the left. But when we read from above, the derivation order first extends to non-terminal.

1. The stack is empty, and we are looking to reduce the rule by S'→S$.
2. Using a "." in the rule represents how many of the rules are already on the stack
3. A dotted item, or simply, the item is a production rule with a dot indicating how much RHS has so far been recognized. Closing an item is used to see what production rules can be used to expand the current structure. It is calculated as follows:

The rules of LR parser as follows.

1. The first item from the given grammar rules adds itself as the first closed set.

2. If an object is present in the closure of the form A→ α. β. γ, where the next symbol after the symbol is non-terminal, add the symbol's production rules where the dot precedes the first item.
3. Repeat steps (B) and (C) for new items added under (B).

LR PARSER ALGORITHM

LR Parsing algorithm is the same for all the parser, but the parsing table is different for each parser. It consists following components as follows.

1. **Input Buffer**
   It contains the given string, and it ends with a $ symbol.
2. **Stack**
   The combination of state symbol and current input symbol is used to refer to the parsing table in order to take the parsing decisions.

PARSING TABLE

Parsing table is divided into two parts- Action table and Go-To table. The **action table** gives a grammar rule to implement the given current state and current terminal in the input stream. There are four cases used in action table as follows.

1. Shift Action- In shift action the present terminal is removed from the input stream and the state ***n*** is pushed onto the stack, and it becomes the new present state.
2. Reduce Action- The number ***m*** is written to the output stream.
3. The symbol ***m*** mentioned in the left-hand side of rule ***m*** says that state is removed from the stack.
4. The symbol ***m*** mentioned in the left-hand side of rule **m** says that a new state is looked up in the goto table and made the new current state by pushing it onto the stack.

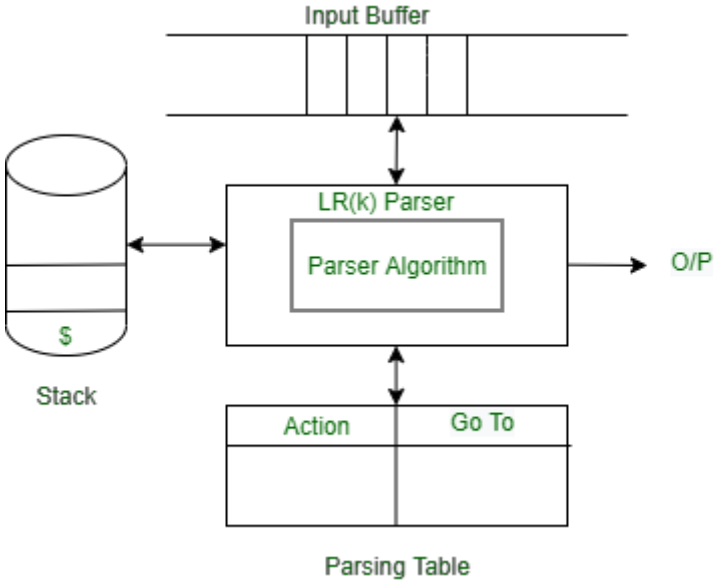
An accept - the string is accepted

No action - a syntax error is reported

Note –

The **go-to table** indicates which state should proceed.

LR parser diagram:



SLR Parser

The SLR parser is similar to LR (0) parser except that the reduced entry. The reduced productions are written only in the FOLLOW of the variable whose production is reduced.

Construction of SLR parsing table –

1. Construct C = { I_0, I_1, ……. I_n}, the collection of sets of LR(0) items for G'.
2. State i is constructed from Ii. The parsing actions for state i are determined as follow

   If [A -> ?.a?] is in I_i and GOTO(I_i , a) = I_j , then set ACTION[i, a] to "shift j". Here a must be terminal.

   If [A -> ?.] is in I_i, then set ACTION[i, a] to "reduce A -> ?" for all a in FOLLOW(A); here A may not be S'.

   Is [S -> S.] is in I_i, then set action[i, $] to "accept". If any conflicting actions are generated by the above rules we say that the grammar is not SLR.

3. The goto transitions for state i are constructed for all nonterminals A using the rule: if GOTO(I_i , A) = I_j then GOTO [i, A] = j.
4. All entries not defined by rules 2 and 3 are made error.

Eg:

If in the parsing table we have multiple entries then it is said to be a conflict.

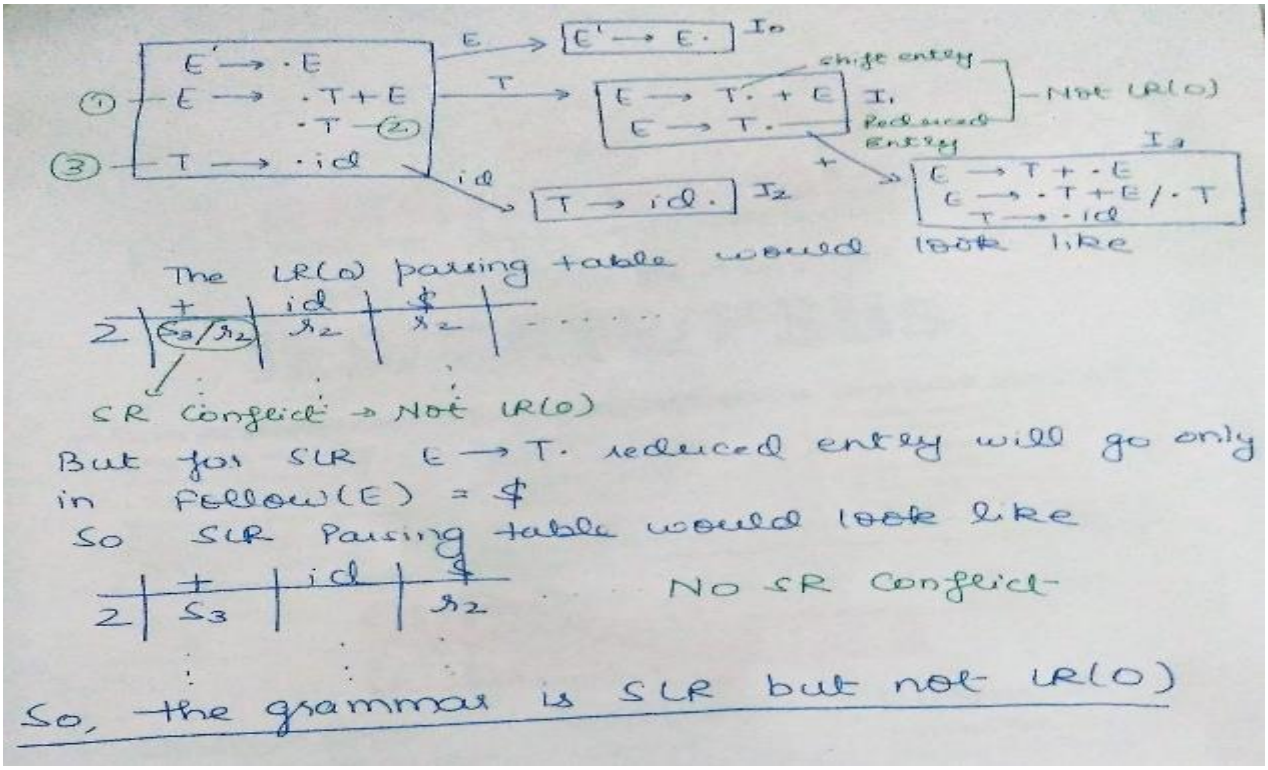
Consider the grammar E -> T+E | T

T ->id

Augmented grammar - E' -> E

E -> T+E | T

T -> id



Note 1 – for GATE we don't have to draw the table, in the GOTO graph just look for the reduce and shifts occurring together in one state. In case of two reductions, if the follow of both the reduced productions has something common then it will result in multiple entries in table hence not SLR. In case of one shift and one reduction, if there is a GOTO operation from that state on a terminal which is the follow of the reduced production than it will result in multiple entries hence not SLR.

Note 2 – Every SLR grammar is unambiguous but there are many unambiguous grammars that are not SLR.

LECTURE NO-8
CONICAL LR AND LALR PARSING TABLE

CLR PARSER

In the SLR method we were working with LR (0) items. In CLR parsing we will be using LR (1) items. LR(k) item is defined to be an item using lookaheads of length k. So, the LR (1) item is comprised of two parts: the LR (0) item and the lookahead associated with the item.

LR (1) parsers are more powerful parser.
For LR (1) items we modify the Closure and GOTO function.

Closure Operation

Closure(I)
repeat
for (each item [A -> ?.B?, a] in I)

for (each production B -> ? in G')

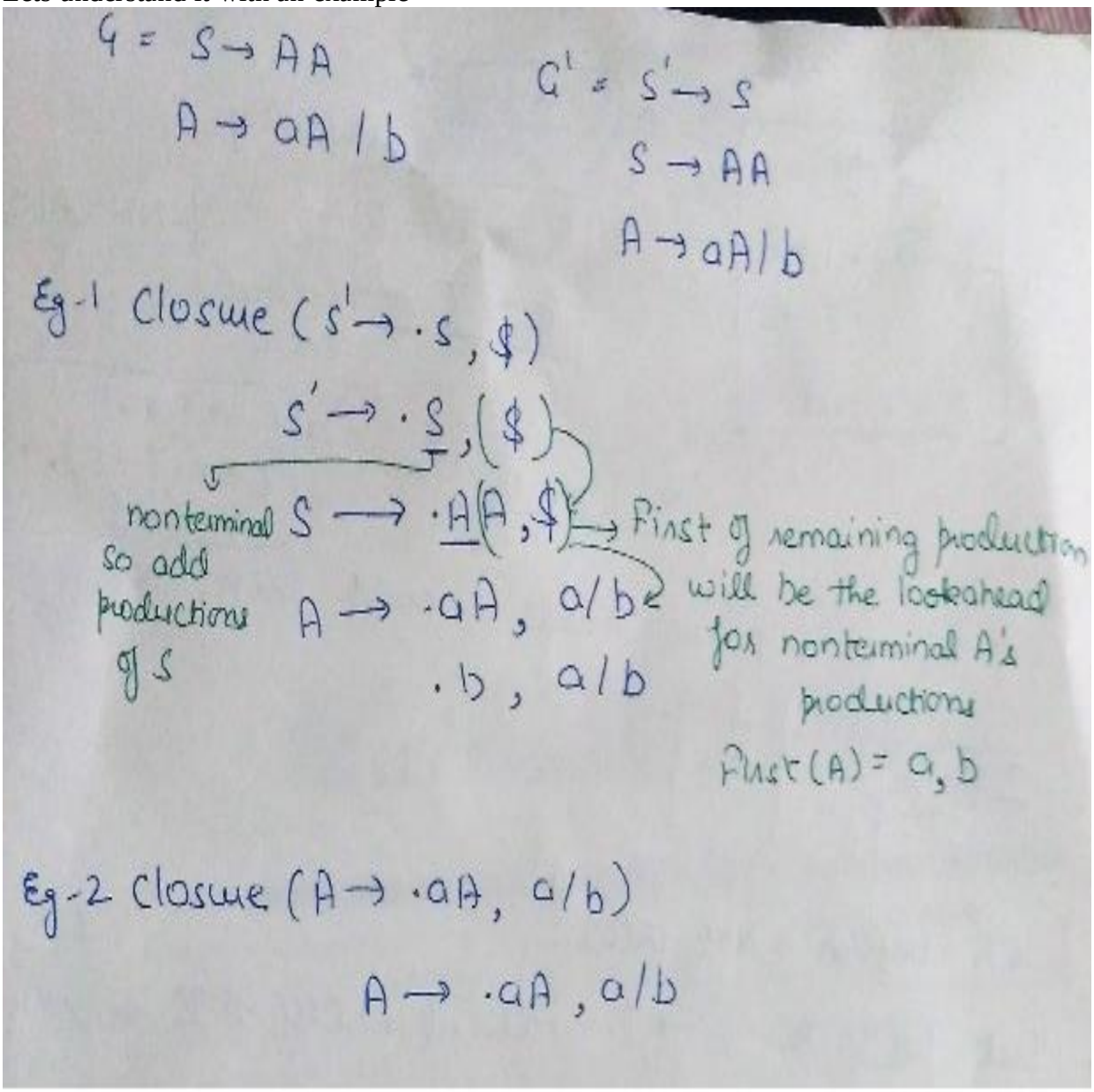
for (each terminal b in FIRST(?a))

add [B -> .? , b] to set I;

until no more items are added to I;

return I;

Lets understand it with an example –



Goto Operation

Goto(I, X)

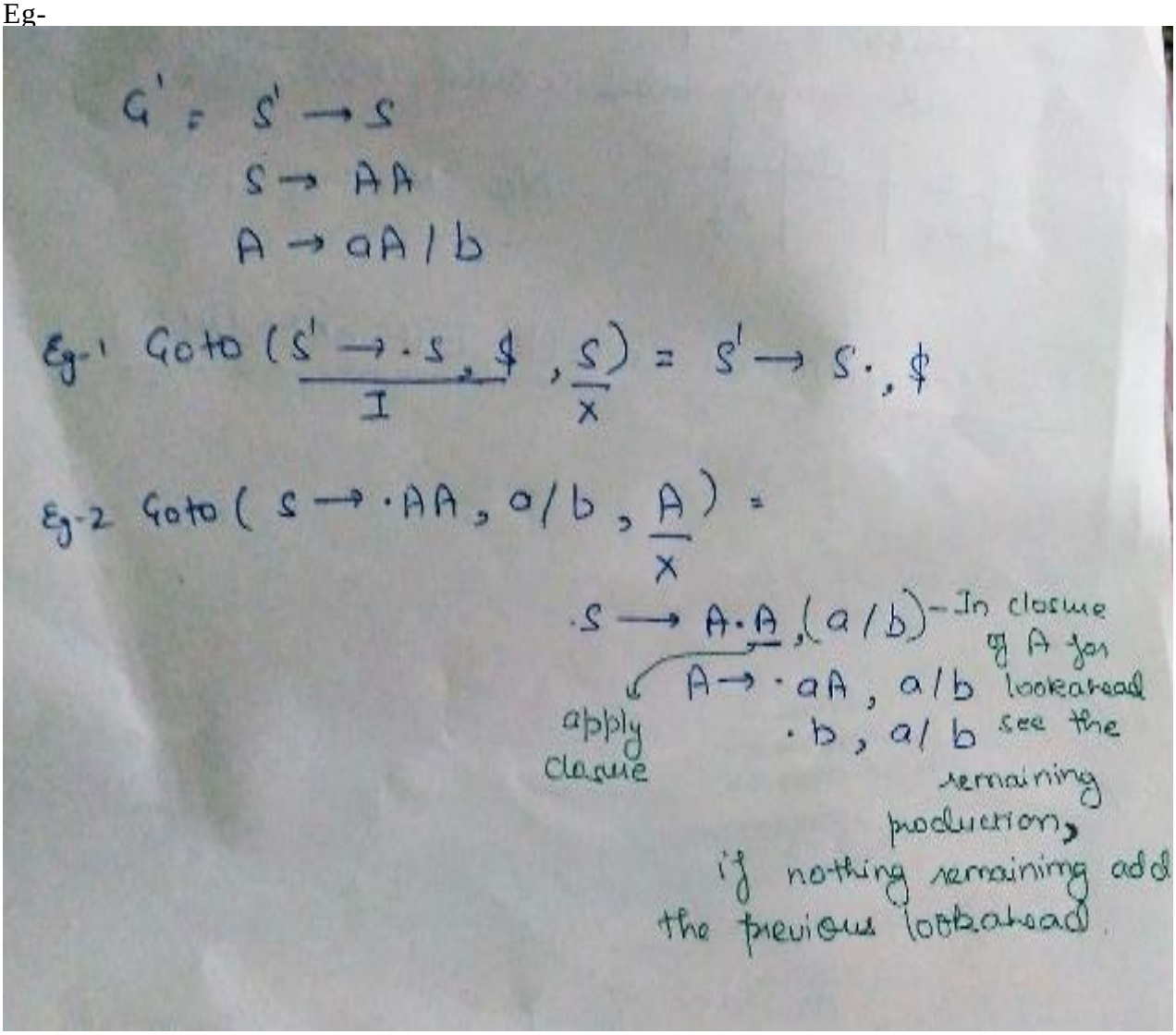
Initialise J to be the empty set;

for (each item A -> ?.X?, a] in I)

Add item A -> ?X.?, a] to se J; /* move the dot one step */

return Closure(J); /* apply closure to the set */

Eg-



LR(1) items

Void items(G')

Initialise C to { closure ({[S' -> .S,

$]})}; Repeat

For (each set of items I in C)

For (each grammar symbol X)

if(GOTO(I, X) is not empty and not in

C) Add GOTO(I, X) to C;

Until no new set of items are added to C;

Construction of GOTO graph

- State I_0 – closure of augmented LR(1) item.
- Using I_0 find all collection of sets of LR(1) items with the help of DFA
- Convert DFA to LR(1) parsing table

Construction of CLR parsing table-

Input – augmented grammar G'

1. Construct C = { I_0, I_1, ……. I_n} , the collection of sets of LR(0) items for G'.
2. State i is constructed from Ii. The parsing actions for state i are determined as follow:

i) If [A -> ?.a?, b] is in I_i and GOTO(I_i , a) = I_j, then set ACTION[i, a] to "shift j". Here a must be terminal. ii) If [A -> ?. , a] is in I_i , A ≠ S, then set ACTION[i, a] to "reduce A -> ?". iii) Is [S -> S. , $] is in I_i, then set action[i, $] to "accept". If any conflicting actions are generated by the above rules we say that the grammar is not CLR.

3. The goto transitions for state i are constructed for all nonterminals A using the rule:
4. if GOTO(I_i, A) = I_j then GOTO [i, A] = j.
5. \

All entries not defined by rules 2 and 3 are made error.

Consider the following grammar

S -> AaAb | BbBa

A -> ?

B -> ?

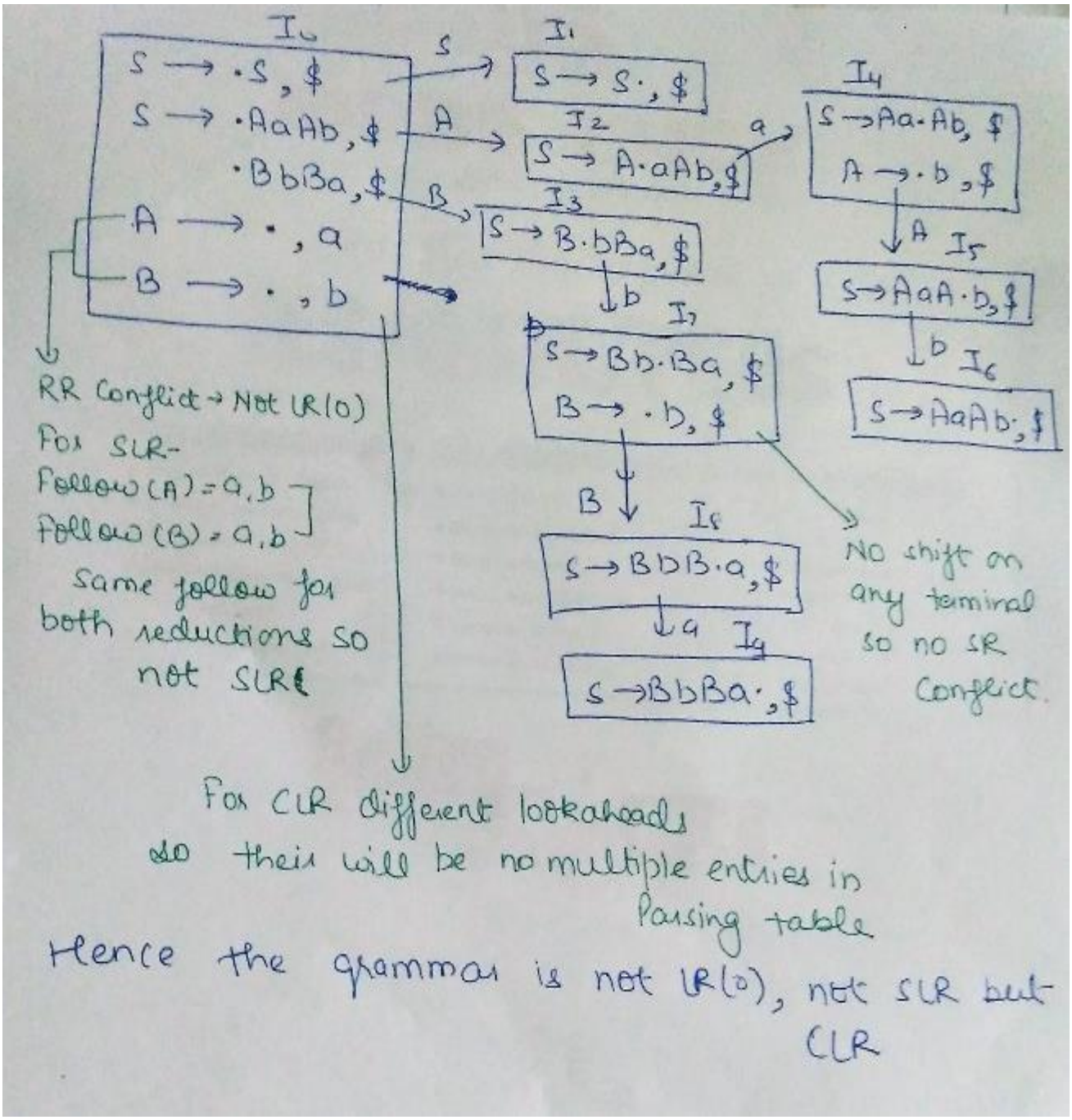
Augmented grammar - S' -> S

S -> AaAb | BbBa

A -> ?

B -> ?

GOTO graph for this grammar will be -



Note – if a state has two reductions and both have same lookahead then it will in multiple entries in parsing table thus a conflict. If a state has one reduction and their is a shift from that state on a terminal same as the lookahead of the reduction then it will lead to multiple entries in parsing table thus a conflict.

LALR PARSER

LALR parser are same as CLR parser with one difference. In CLR parser if two states differ only in lookahead then we combine those states in LALR parser. After minimisation if the parsing table has no conflict that the grammar is LALR also.

Eg:

consider the grammar S -

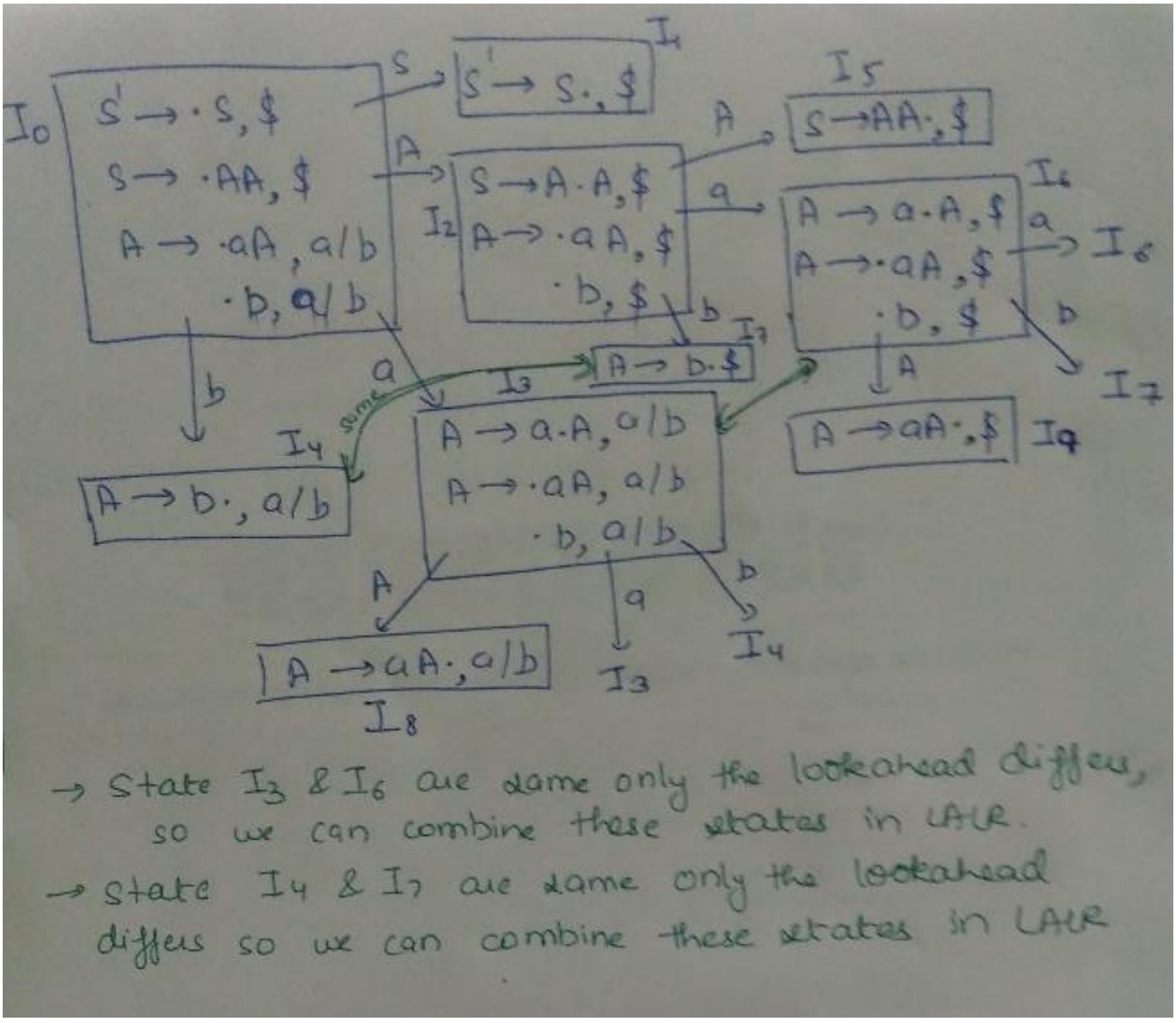
A -> aA | b

Augmented grammar

S' -> S

S ->AA

A -> aA | b



LECTURE NO-9
PARSING WITH AMBIGUOUS GRAMMAR

Context **F**ree **G**rammars (CFGs) are classified based on:

- Number of Derivation trees
- Number of strings

Depending on Number of Derivation trees, CFGs are sub-divided into 2 types:

- Ambiguous grammars
- Unambiguous grammars

Ambiguous grammar:

A CFG is said to ambiguous if there exists more than one derivation tree for the given input string i.e., more than one **L**eft **M**ost **D**erivation **T**ree (LMDT) or **R**ight **M**ost **D**erivation **T**ree (RMDT).

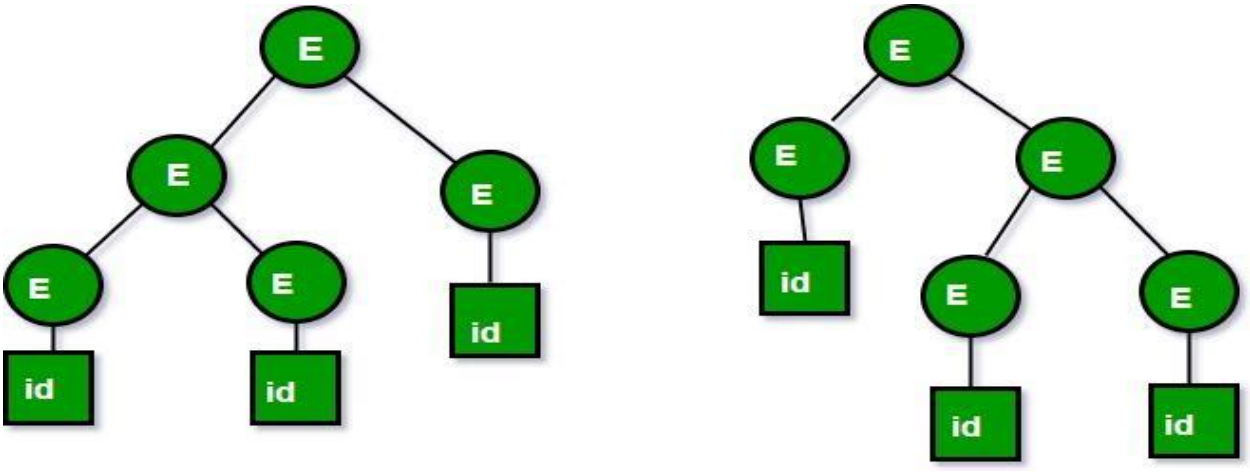
Definition: G = (V, T, P, S) is a CFG is said to be ambiguous if and only if there exist a string in T* that has more than on parse tree.

For Example:

1. Let us consider this grammar: **E -> E+E|id**

We can create 2 parse tree from this grammar to obtain a string **id+id+id** :

The following are the 2 parse trees generated by left most derivation:



Both the above parse trees are derived from same grammar rules but both parse trees are different. Hence the grammar is ambiguous.

2. Let us now consider the following grammar:

Set of alphabets ∑ = {0,…,9, +, *, (,)}

E -> I

E -> E + E

E -> E * E

E -> (E)

I -> ε | 0 | 1 | … | 9

From the above grammar String **3*2+5** can be derived in 2 ways:

I) First leftmost derivation

E=>E*E
=>I*E
=>3*E+E
=>3*I+E
=>3*2+E
=>3*2+I
=>3*2+5

II) Second leftmost derivation

E=>E+E
=>E*E+E
=>I*E+E
=>3*E+E
=>3*I+E
=>3*2+I
=>3*2+5

Following are some examples of ambiguous grammars:

- S-> aS |Sa| Є
- E-> E +E | E*E| id
- A -> AA | (A) | a
- S -> SS|AB , A -> Aa|a , B -> Bb|b

Whereas following grammars are unambiguous:

- S -> (L) | a, L -> LS | S
- S -> AA , A -> aA , A -> b

Inherently ambiguous Language:

Let L be a Context Free Language (CFL). If every Context Free Grammar G with Language L = L(G) is ambiguous, then L is said to be inherently ambiguous Language. Ambiguity is a property of grammar not languages.

Ambiguous grammar is unlikely to be useful for a programming language, because two parse trees structures (or more) for the same string(program) implies two different meanings (executable programs) for the program.

An inherently ambiguous language would be absolutely unsuitable as a programming language, because we would not have any way of fixing a unique structure for all its programs.

For example,

L = {anbncm} ∪ {anbmcm}

Note : Ambiguity of a grammar is undecidable, i.e. there is no particular algorithm for removing the ambiguity of a grammar, but we can remove ambiguity by:

Disambiguate the grammar

rewriting the grammar such that there is only one derivation or parse tree possible for a string of the language which the grammar represents.

Context **F**ree **G**rammars (CFGs) are classified based on:

- Number of Derivation trees
- Number of strings

Parsing ambiguous grammars using LR parser

LR parser can be used to parse ambiguous grammars. LR parser resolves the conflicts (shift/reduce or reduce/reduce) in parsing table of ambiguous grammars based on certain rules (precedence and/or associativity of operators) of the grammar.

Example:

Let's take the following ambiguous grammar:

E -> E+E

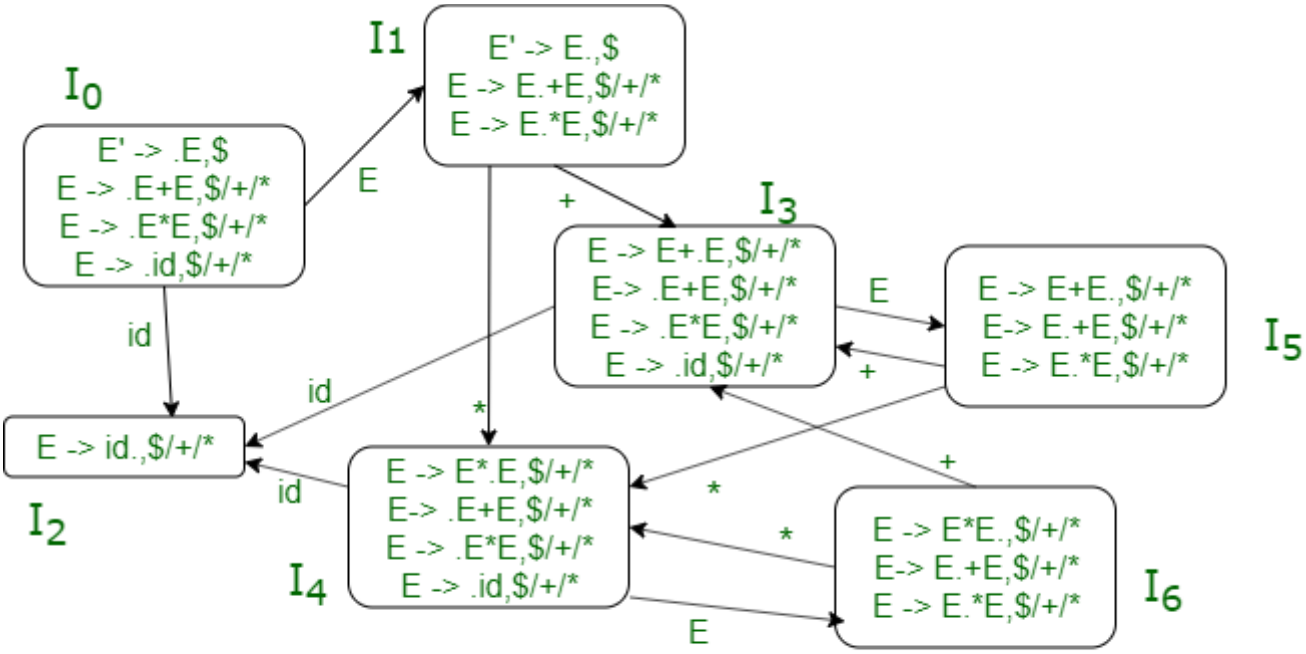
E -> E*E

E -> id

Let's assume, the precedence and associativity of the operators (+ and *) of the grammar are as follows:

- "+" and "*" both are left associative,
- Precedence of "*" is higher than the precedence of "+".

If we use LALR (1) parser, the LR (1) item DFA will be:



LR(1) item DFA

From the LR(1) item DFA we can see that there are shift/reduce conflicts in the state I_5 and I_6. So the parsing table is as follows:

	id	+	*	$	E
I_0	S_2				1
I_1		S_3	S_4	accept	
I_2		r_3	r_3	r_3	
I_3	S_2				5
I_4	S_2				6
I_5		r_1/S_3	r_1/S_4	r_1	
I_6		r_2/S_3	r_2/S_4	r_2	

Parsing Table

There are both shift and reduce moves in I_5 and I_6 on "+ and "*". To resolve this conflict, that is to determine which move to keep and which to discard from the table we shall use the precedence and associativity of the operators.

Consider the input string:

id + id + id

Let's look at the parser moves till the conflict state according to the above parsing table.

Stack	Symbol	Input
0		id + id + id $
0 2	id	+ id + id $
0 1	E	+ id + id $
0 1 3	E +	id + id $
0 1 3 2	E + id	+id $
0 1 3 5	E + E	+id $ reduce
0 1	E	+id $

Parser 1

Stack	Symbol	Input
0		id + id + id $
0 2	id	+ id + id $
0 1	E	+ id + id $
0 1 3	E +	id + id $
0 1 3 2	E + id	+id $
0 1 3 5	E + E	+id $ Shift
0 1 3 5 3	E + E +	id $

Parser 2

1.	If we take the reduce move of I_5 state on symbol "+" as in parser 1, then the left "+" of the input string is reduced before the right "+", which makes "+" left associative.
2.	If we take the shift move of I_5 state on symbol "+" as in parser 2, then the right "+" of the input string is reduced before the left "+", which makes "+" right associative.
	Similarly, taking shift move of I_5 state on symbol "*" will give "*" higher precedence over "+", as "*" will be reduced before "+". Taking reduce move of I_5 state on symbol "*" will give "+" higher precedence over "*", as "+" will be reduced before "*". Similar to I_5, conflicts from I_6 can also be resolved.

According to the precedence and associativity of our example, the conflict is resolved as follows,

- The shift/reduce conflict at I_5 on "+" is resolved by keeping the reduce move and discarding the shift move, which makes "+" left associative.

- The shift/reduce conflict at I_5 on "*" is resolved by keeping the shift move and discarding the reduce move, which will give "*" higher precedence over "+".
- The shift/reduce conflict at I_6 on "+" is resolved by keeping the reduce move and discarding the shift move, which will give "*" higher precedence over "+".
- The shift/reduce conflict at I_6 on "*" is resolved by keeping the reduce move and discarding the shift move, which makes "*" left associative.

Generally, the parser generator tool **YAAC** resolves conflicts due to ambiguous grammars as follows,

- Shift/reduce conflict in the parsing table is resolved by giving priority to shift move over reduce move. If the string is accepted for shift move, then reduce move is removed, otherwise shift move is removed.
- Reduce/reduce conflict in the parsing table is resolved by giving priority to first reduce move over second reduce move. If the string is accepted for first reduce move, then second reduce move is removed, otherwise first reduce move is removed.

LECTURE NO-10
AUTOMATIC PARSER GENERATOR

Introduction to YACC

A parser generator is a program that takes as input a specification of a syntax, and produces as output a procedure for recognizing that language.

Historically, they are also called compiler-compilers.

YACC (yet another compiler-compiler) is an LALR (1) (Look Ahead, Left-to-right, Rightmost derivation producer with 1 lookahead token) parser generator. YACC was originally designed for being complemented by Lex

.

Input File:

YACC input file is divided into three parts.

/* definitions */

....

%%

/* rules */

....

%%

/* auxiliary routines */

....

Input File:

Definition Part:

- The definition part includes information about the tokens used in the syntax definition:

%token NUMBER

%token ID

- Yacc automatically assigns numbers for tokens, but it can be overridden by

%token NUMBER 621

- Yacc also recognizes single characters as tokens. Therefore, assigned token numbers should not overlap ASCII codes.
- The definition part can include C code external to the definition of the parser and variable declarations, within **%{** and **%}** in the first column.
- It can also include the specification of the starting symbol in the grammar:

%start nonterminal

- **RULE PART**
- The rules part contains grammar definition in a modified BNF form.
- Actions is C code in { } and can be embedded inside (Translation schemes).